

Engineering Operation Procedures

Normain Engineering Operation Procedures

Purpose and Scope

This document establishes standardized engineering operational procedures to ensure consistent, reliable, and secure delivery of the Normain service. Its primary objective is to align engineering practices toward operational excellence by enforcing clarity, repeatability, and accountability across technical processes.

It applies to the entire engineering organization and governs activities related to the development, deployment, and ongoing operation of the Normain service.

Covered environments include all stages of the software lifecycle—development, staging, and production.

Development Process

GitHub Flow

We follow GitHub Flow for all development work:

- All development tasks start by creating a branch from main.
- All code changes are done in that branch, keeping commits focused and clear.
- When commits are done, open a pull request so others can review your code and automated checks can run.
- Once the PR is approved and tests pass, merge the branch back into main.

Issue tracking

We use Linear to manage development work by maintaining a backlog of work, prioritizing upcoming work and to track ongoing activity.

By default, all work starts from a Linear issue. If there is no existing ticket for your intended work, create one with some level of description.

Branch naming

Branch should always be the ones proposed by Linear for the issue you are working on.

Other projects may let work descriptions be part of the branch names (e.g. feature/fix/infra etc), but we capture this type of metadata in Linear instead.

Code Reviews

When a PR is marked Ready for Review, it is reviewed by at least one other developer.

In general PR reviews are self-assigned by the reviewer. Reviewers are accountable for upholding codebase quality. At the early stage of the company, most PRs will tend to get reviewed by Kalle (CTO). However as new developers learn the codebase, we will actively work towards the goal that anyone is able to review any PR. The review is where the team takes ownership of the code that was created by an individual.

Discussion around a PR is done using the PR comment feature. When a review is done, the reviewer may require changes from the PR creator. The creator will then address the comments and make changes and ask for a re-review.

When a PR has been Approved, the creator closes the PR by merging the code.

In the cases where it's useful to create a PR before it is ready for review, create a draft PR. This will enable linting checks, run tests and enable testing of CI/CD changes.

All changes merged to main require at least one approving review from someone other than the author. There are no exceptions to this requirement.

Emergency Fixes

Emergency situations such as security issues or critical functionality bugs may require an engineer to work against a sharp timeline. The review requirement still applies — no change reaches production without an independent review — but the process is expedited:

- If a Linear issue is missing, create a branch fix/{{ title }} and backfill the issue.
- Reviews are handled synchronously and given top priority; a reviewer is paged if none is immediately available.

Once the emergency is resolved, any expedited tracking steps are backfilled to restore full traceability.

QA and Testing

Normain's QA and testing practices are designed to ensure code quality, system reliability, and compliance with security and privacy requirements across all environments. The following layers collectively define our testing framework. We are actively working toward a target state where all complex code paths are covered by automated tests, reducing the risk of regressions and increasing confidence in system changes:

- **Unit Tests**
Every code module must include unit tests that validate individual functions or components (including UI components) in isolation. Unit tests ensure correctness of logic and facilitate early defect detection during development.
- **Integration Tests**
Integration tests validate interactions between system components and services. These tests cover API contracts, data flows, and system boundaries, ensuring components work together as intended.
- **End-to-End (E2E) Tests**
E2E tests exercise full user journeys and critical business processes across the entire stack. E2E testing ensures that the system operates as expected in scenarios resembling real-world use.

Test Data Management

- All test data seeding is performed using Normain's proprietary seeding solution, which enables deterministic and repeatable test scenarios across environments.
- Under no circumstances shall production data be used in development or testing environments, to maintain data privacy and regulatory compliance. We keep entire deployment architectures fully separate (e.g. no shared networks or database servers) to avoid accidental misconfiguration or otherwise intended data sharing / copying.

AI-Driven Algorithm Quality

As Normain's products increasingly rely on AI-driven capabilities, algorithm quality is assessed as part of QA. Planned: a formal evaluation framework to:

- Define quantitative metrics for assessing algorithmic performance, fairness, and stability.
- Monitor changes in model behavior over time.
- Support rapid iteration while maintaining trust and reliability in AI outputs.

This framework will evolve to become an integral part of our QA process, ensuring our AI features meet the same rigorous standards as the rest of our software systems.

Feature Flags

All non-trivial changes to application behavior, user experience, or system interactions shall be protected by feature flags. Implementing feature flags is mandatory for changes with potential impact on stability, performance, or user outcomes.

Flags are to be reviewed regularly and retired once they are no longer necessary to prevent unnecessary complexity and technical debt. Emergency procedures for disabling feature flags must exist and be known to relevant engineering and operations personnel, ensuring immediate mitigation capability for issues arising in production.

Feature flags are not yet in active use. Planned: adopt a flag-management capability (e.g. PostHog or equivalent) to provide rollout control, tracking, and analytics as the system scales.

As the system scales and complexity grows, feature flag management will become increasingly critical to maintain deployment agility and system reliability.

Environments

We maintain the following environments:

Environment	Description
Production	- Manual CI triggering
Staging (this is the current “Dev” env)	- Manual CI trigger - Purpose: having a place to test everything right before production. We can also make urgent fixes and test towards this env. - Should be configured as similar to production as possible.
Dev (will not be used currently)	- Builds on each main commit - Purpose: To always have a live env with the latest features. - Early detection of deployment issues.

prx (pr1, pr2, ...)	- Manual CI trigger - Up for grab for anyone who wants to test anything - infra or features. - Should reset the database for each deployment.
---------------------	---

Staging Requirements

All production-bound changes must pass through a staging environment that mirrors production configurations and scale as closely as feasible. Pre-production validations include integration tests, performance benchmarks, and fault injection where applicable.

Deployment Process

Deployments follow a structured, versioned process through CI/CD pipelines with stage-specific validations.

- **Pipeline Stages:** Code checkout, build, test (unit, integration), package and deploy to an environment.
- **Control Mechanisms:** Automated deployments are default; manual gates are enforced for high-risk changes, SEV-related remediations, or policy exceptions.
- **Progressive Deployment Options:** Canary and blue-green strategies are available to reduce blast radius. Rollouts must be observable, and health metrics must inform promotion or rollback decisions.
- **Rollback and Hotfixes:** Rollback is mandatory on sustained degradation or failed health checks unless feature flags or other remediation actions can be taken. Hotfixes must follow a minimal diff policy, reviewed for risk, and prioritized through incident channels.

Monitoring and Alerting

Metrics and Objectives

Each service must define and maintain Service Level Indicators (SLIs), Service Level Objectives (SLOs), and where applicable, Service Level Agreements (SLAs).

SLIs must capture core user-facing metrics such as latency, availability, error rates, and saturation.

SLOs define acceptable thresholds; breach tracking is mandatory and feeds into reliability reviews.

Observability Tooling

Monitoring is primarily centralized to Azure metrics dashboards.

Logs are retained and queryable via structured logging systems in Azure and Betterstack.

Alerting Rules

Alerts must be actionable, deduplicated, and tied to service SLOs. Each alert must have a defined owner, documented threshold, and escalation path. Paging is reserved for high-severity events impacting user experience or critical infrastructure.

Runbooks

Every alert and critical operational task should be backed by a current runbook. Runbooks include triage steps, diagnostic commands, mitigation procedures, and escalation contacts. Ownership and periodic review are enforced as part of operational hygiene.

Incident Management

Severity Levels

Incidents are classified by impact and urgency using a standardized severity scale:

- **Critical:** Critical user impact or full system outage. Immediate, all-hands response.
- **High:** Major functionality degradation with significant user impact. Rapid coordinated response.
- **Low:** Limited degradation or partial feature impairment with workaround. Monitored resolution.

Severity level must be assigned at incident initiation and updated as context evolves.

On-Call Rotation

Engineering teams maintain 24/7 on-call coverage with a published rotation schedule and enforced handoff protocol. Each on-call shift includes documented expectations, escalation procedures, and access to required tooling and credentials. Backup coverage is mandatory.

Incident Workflow

- **Detection:** Triggered via monitoring systems, user reports, or automated anomaly detection.
- **Acknowledgement:** On-call engineers must acknowledge alerts within defined SLA windows.
- **Containment:** Immediate action to prevent further user impact or data loss. Temporary fixes allowed.
- **Communication:** Internal coordination via incident channels; external updates issued for Critical and High incidents as needed. All communication is timestamped and archived.
- **Resolution:** Root issue addressed, system restored, and verified healthy.
- **Postmortem:** Required for Critical and High issues. Includes root cause analysis, timeline reconstruction, impact summary, and specific action items with owners and deadlines. Blameless culture is enforced; recurrence prevention is the primary goal.

Tooling and Vendor Management

Normain has taken a bet on the following main external tools and services to ensure technical coherence, operational efficiency, and cost-effectiveness across the engineering organization.

We are always open to new tools that will improve our development process.

In addition to the below, we also use other tooling on a more individual level as long as they have been approved in advance. Approved tooling is available in the [Vendor Master Sheet](#).

We are always open to new tools that will improve our development process, but there will need to be a risk analysis done before adoption.

GitHub

GitHub is a core component for the Normain development process and covers:

- Source code versioning (Repository)
- CI/CD (GitHub Actions)
- Security/vulnerability scanning and issue handling (GitHub Advanced Security)

Infrastructure as Code (IaC)

All infrastructure is provisioned and managed using Terraform, with Azure as the primary cloud provider. IaC modules must follow approved patterns, use version control, and undergo peer review. State is centrally managed with access controls and audit logging enabled. Changes must be applied through CI/CD pipelines to ensure consistency and traceability.

Configuration and Secrets Management

Configuration values and environment variables are managed per-environment, with clear separation between development, staging, and production. Secrets are stored and managed securely in 1Password, with role-based access and rotation policies enforced. Direct embedding of secrets in code or configuration files is strictly prohibited.

Cloud Providers

We anticipate customer requests to be able to deploy Normain on any of the major cloud providers. We therefore keep the set of cloud features we use limited (e.g. containers, managed Postgres databases, load balancers) and avoid using features that are cloud provider specific (e.g. serverless code, proprietary databases).

DNS

We use CloudFlare for DNS management.

Security Vulnerabilities

Normain uses the following mechanisms to keep security vulnerabilities out:

- GitHub Dependabot, for monitoring security issues with node libraries we make ourselves dependent on.
- Gype for vulnerability scanning of the container images we build in our CI/CD pipeline. The pipeline will fail if a critical issue is found, otherwise issues will be reported

Vulnerability Management

GitHub Security issues need to be monitored by the development team. When an issue has been created, create a Linear ticket with the “Security” label, linking to the GitHub Security issue.

Prioritize, assign and fix the Linear issue the same way other issues are handled.

Manually Identified Vulnerabilities

There will always be vulnerabilities that don’t have automated checks. If an issue is identified, create a separate product issue (NOR) with the “Security” label to directly fix the vulnerability when the fix is apparent to you, or a Security Event Report (INT) if you are unsure what the best remediation is.

Documentation

We use English for any documentation created.

Source code

The primary source of documentation for our codebase is the code itself. Code should be written in a clear, idiomatic style that conveys intent without requiring external explanations.

When logic, algorithms, or system interactions become non-obvious or complex, comments shall be added directly in the code to explain purpose, assumptions, or decisions that are not self-evident from the implementation.

No README or external documentation files shall be created unless a specific need is identified, such as onboarding requirements, high-level architectural overviews, or operational instructions that cannot reasonably reside in code or inline comments. Until such a need arises, teams will refrain from maintaining separate README files to avoid unnecessary duplication and divergence from the actual code behavior.

Non-source code

Documentation that is not closely source code-related is put in Normain's Google Workspace.

Document History

Version	Date	Description	Written by	Approved by
1.0.0	2025-06-08	Initial version (uploaded as Vanta evidence)	Gustav Lidén	Kalle Hansson
1.1.0	2026-06-16	Removed draft appendix; required review for all merges; added document control	Kalle Hansson	Kalle Hansson